

Università degli studi di Urbino
Carlo Bo

Corso di Laurea in Informatica Applicata

Corso di Programmazione e
Modellazione a oggetti

Anno Accademico 2025/2026

Studentessa: Chiara La Paglia
Matricola: 315111

Docente: Sara Montagna

Indice

1 Analisi	3
1.1 Requisiti.	3
1.2 Modello del dominio	6
2 Design	8
2.1 Architettura	8
2.1.1 Model	8
2.1.2 View	8
2.1.3 Controller	9
2.2 Design dettagliato	9
3 Sviluppo	13
3.1 Testing automatizzato	13
3.2 Metodologia di lavoro	13
3.3 Note di sviluppo	14
4 Commenti finali	15
4.1 Autovalutazione e lavori futuri	15
5 Guida utente	16
5.1 Menu principale	16
5.2 Inizio partita	17
5.3 Soluzione	18
5.4 Fine partita	20

Capitolo 1

Analisi

L'obiettivo del progetto è lo sviluppo di un applicativo per giocare ad una variante di "Panic Lab", un gioco di velocità, a cui possono partecipare diversi giocatori, con lo scopo di trovare l'ameba scappata dal laboratorio. Questo gioco è composto da 25 carte disposte in cerchio in maniera casuale: le carte sono in parte delle amebe con diversi colori, forme e fantasie, le restanti sono delle carte speciali. Ci sono poi dei dadi che vengono lanciati a inizio turno che servono per determinare il tipo di ameba che va trovata, da dove far partire la ricerca e se bisogna spostarsi in senso orario o antiorario. Una volta trovata la partenza giusta nel tabellone si comincia a cercare la carta con l'ameba che ha le caratteristiche designate dai dadi, che potranno essere modificate durante la ricerca se ci si imbatte in una carta di tipo mutazione, questa può far cambiare una delle 3 caratteristiche dell'ameba; l'ultima cosa a cui fare attenzione durante il gioco sono le grate: quando se ne incontra una bisogna saltare tutte le carte a seguire finché non si ritrova un'altra carta di quel tipo, a quel punto si continua a cercare normalmente.

C'è un'eccezione nel trovare il giusto risultato perché, se ci si trova a passare due volte per la stessa identica carta mutazione, allora si dice che l'ameba "muore" in quel punto e quindi la soluzione non sarà una delle carte base con le amebe, ma la carta mutazione in cui ci si è imbattuti per la seconda volta. Questo metodo evita la creazione di loop infiniti all'interno del gioco.

1.1 Requisiti

L'applicazione permette di scegliere quanti giocatori far partecipare e quanti turni giocheranno. Una volta fatte queste scelte verrà avviato il gioco vero e proprio: si vedranno le carte distribuite nello schermo secondo uno schema circolare, tenendo le carte lungo i bordi di un

tabellone. A questo punto verranno visualizzati anche i dadi con i rispettivi risultati e partirà il timer per il primo giocatore. Una volta che il giocatore avrà scelto una carta da selezionare come soluzione dell'enigma, se la carta è quella corretta, verrà salvato il tempo, altrimenti verrà applicata una penale sul tempo impiegato, in entrambi i casi si passerà al turno del giocatore successivo (se si sta giocando in modalità multiplayer, se così non fosse partirà il turno seguente per la sfida personale).

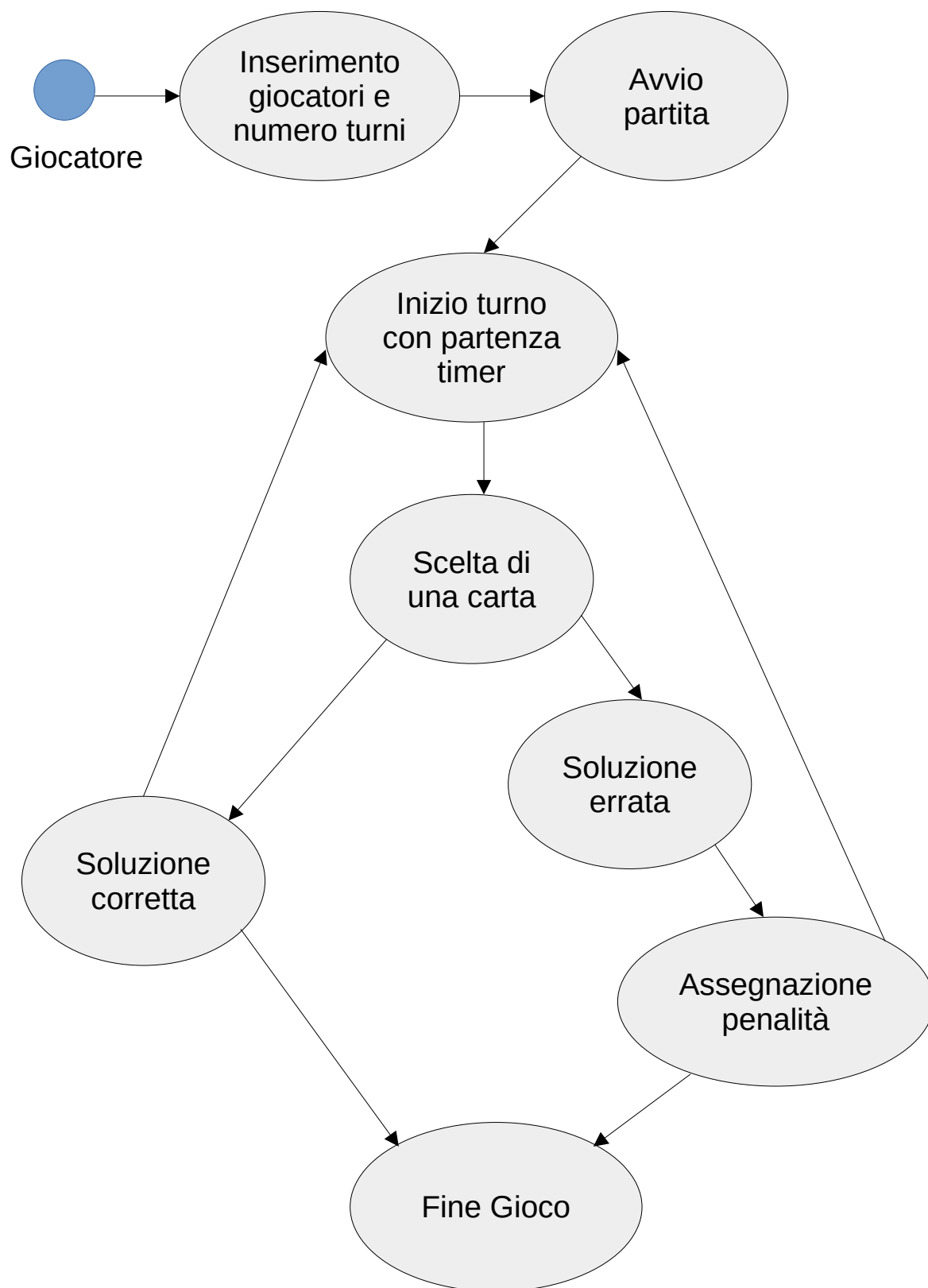


Figura 1.1: Diagramma dei casi d'uso

1.2 Modello del dominio

Nella modalità di gioco che si andrà a sviluppare, invece che avere tutti i partecipanti che giocano contemporaneamente, si utilizzerà una versione a tempo per cui ogni giocatore avrà dei dadi diversi per il proprio turno e il vincitore sarà colui che avrà impiegato meno tempo sommando i risultati di tutti i turni.

I giocatori verranno identificati dall'entità *Giocatore* a cui potranno assegnare il proprio nome se vorranno, altrimenti sarà il gioco a distinguerli come "Giocatore N".

Allo stesso modo il gioco avrà un settaggio automatico di un numero di turni e giocatori standard nel caso in cui l'input inserito dai partecipanti non sia coerente con quello che ci si aspetta.

Avremo poi a comporre il gioco un tabellone con le caselle in cui verranno inserite le carte, queste ultime potranno essere di tipo base (le amebe) o di tipo speciale (mutazione, grata, partenza).

I dadi saranno a due facce per quanto riguarda la forma (lumaca, fantasma), la fantasia (pois, strisce), il colore (viola, arancione) e il senso, che in questa modalità sarà modellato come un dado in più rispetto al gioco originale in cui si capiva tramite il dado che segnalava il laboratorio di partenza. Oltre a questi dadi ne avremo un ultimo a 3 facce che indicherà il colore del laboratorio da cui partire (rosso, blu, giallo).

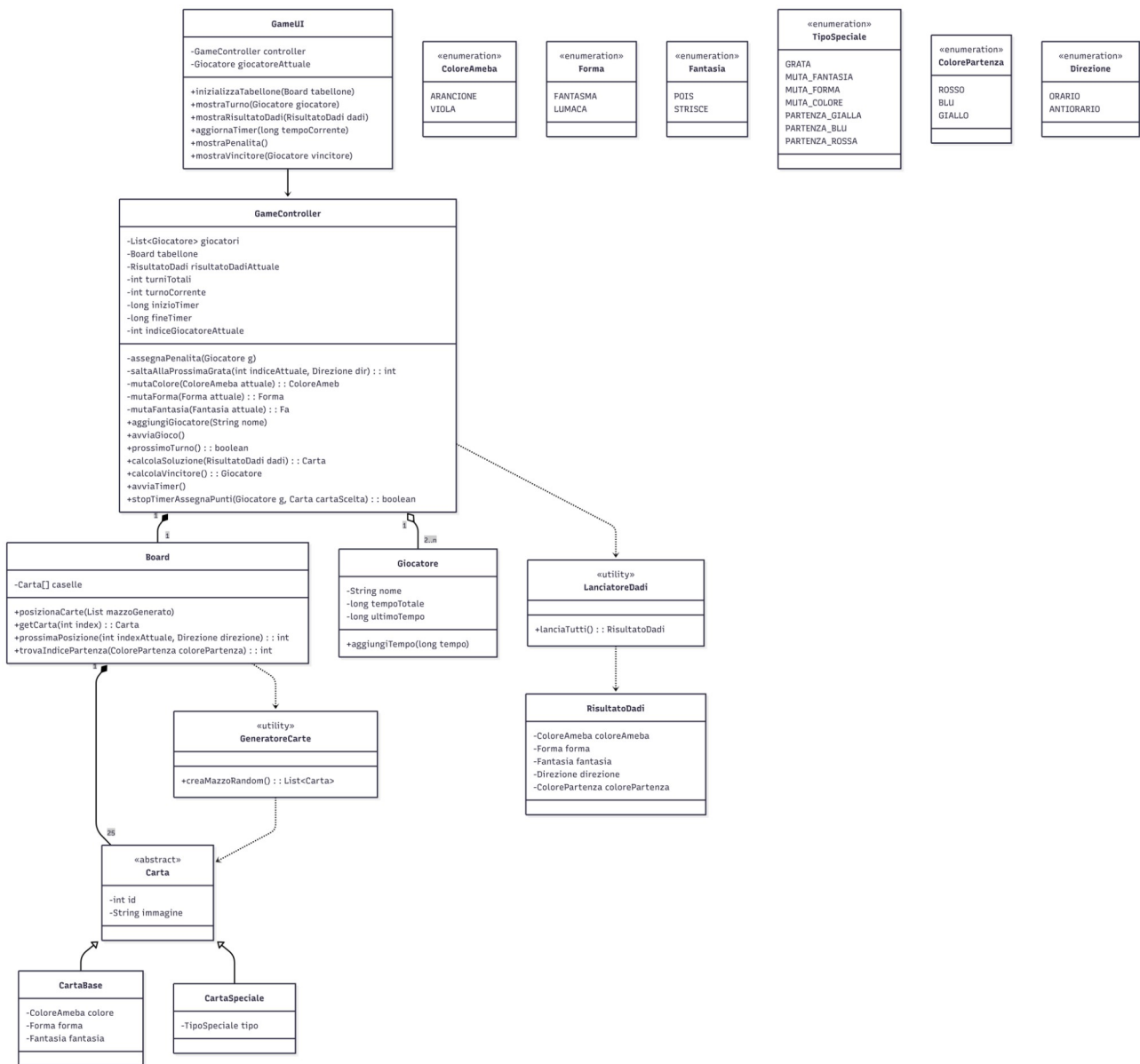


Figura 1.2: Schema UML dell’analisi del problema, con rappresentate le entità ed i rapporti tra loro

Capitolo 2

Design

2.1 Architettura

Per lo sviluppo dell'architettura di PanicLab si è deciso di seguire il pattern architetturale MVC (Model - View - Controller).

2.1.1 Model

Il *Model* si occupa di gestire le metodologie di accesso e modifica dei dati dell'applicazione e del dominio dell'applicativo, determinando coerentemente come le interazioni dell'utente influenzino lo stato corrente dell'applicazione. Il *Model* del sistema sarà composto da diverse entità che interagiscono tra loro. Lo scopo del *Model* sarà fornire al *Controller* accesso allo stato attuale dell'applicazione.

Per fornire tale funzionalità ci sono le entità *Giocatore*, *Board* e *RisultatoDadi* che vengono passate al *Controller*, in questo modo il controller potrà conoscere lo stato attuale dell'applicazione. A sua volta l'entità *Board* avrà al suo interno la configurazione delle entità *Carta* nel tabellone.

2.1.2 View

La *View* consiste nella schermata dell'applicazione che si occupa della gestione della parte grafica, della User Experience e dell'interazione con l'utente ed è gestita interamente dall'entità *GameUI* tramite l'utilizzo della libreria Java Swing.

È compito della *View* registrare ed informare il *Controller* di interazioni con l'applicazione da parte dell'utente, in attesa di una sua risposta sul cambiamento dei dati.

L'entità *Controller* è disaccoppiata dalla parte della *View*, questa indipendenza delle due parti permette di cambiare la parte grafica modificando solamente la *View* e lasciando invariati *Model* e *Controller*.

2.1.3 Controller

Il *Controller*, gestito nella sua totalità dall'entità *GameController*, è il componente a cui spetta gestire le interazioni da parte dell'utente nella *View*, comunicando quindi al *Model* le scelte prese dall'user. Una volta che il *Model* avrà completato l'elaborazione della richiesta di cambiamento, il *Controller* avviserà la *View*, in modo tale che quest'ultima possa aggiornarsi in maniera coerente secondo le regole specificate dal *Model*.

2.2 Design dettagliato

Gestione della Board

- Problema:

Il gioco richiede un tabellone con delle caselle posizionate in maniera circolare lungo i bordi della *Board*. Le carte devono essere sistemate ad ogni partita in maniera casuale ed è necessario gestire le carte speciali, come le carte che indicano le diverse partenze o le carte che mutano l'ameba cercata.

- Soluzione:

Per affrontare questo problema verrà creata innanzitutto una struttura gerarchica di classi per la gestione delle carte. La classe base all'interno di questa gerarchia sarà denominata *Carta*. I sottotipi di questa classe saranno di tipo *CartaBase* e *CartaSpeciale*, queste sfrutteranno il concetto di ereditarietà e grazie a questo polimorfismo la *Board* potrà fare operazioni considerando l'identità *Carta* senza sapere prima quale delle due le verrà data in pasto.

Inoltre verrà utilizzata una classe utility, *GeneratoreCarte*, che semplificherà la logica della Board perché si occuperà della

creazione del mazzo con l'aggiunta di tutte le carte e il mescolamento delle stesse.

L'entità *Board* permetterà di identificare la posizione della partenza segnalata dal lancio dei dadi, trovare una carta in base all'indice, trovare l'indice della prossima posizione in base al senso (orario o antiorario) dato dal lancio dei dadi e posizionare le carte in maniera molto semplice grazie all'utilizzo di *GeneratoreCarte* che, chiamato dal *Controller*, creerà il mazzo che in seguito verrà passato alla *Board* sempre dal *Controller*.

Gestione dei Dadi

- Problema:

In questo gioco avremo 5 diversi dadi lanciati, 3 per identificare l'ameba e gli ultimi due per indicare la casella di partenza e il senso di gioco.

- Soluzione:

Per questo verrà creata una classe utility chiamata *LanciatoreDadi*, che si occuperà di estrarre randomicamente una delle facce per ogni dado, i risultati verranno poi salvati come dati all'interno della classe *RisultatoDadi*, chiamata dal *Controller* per fare le giuste ispezioni nel momento in cui il giocatore selezionerà una carta. *LanciatoreDadi* risolverà in maniera efficace il problema dell'estrazione andando ad utilizzare un metodo comune per ogni dado, questo metodo infatti sarà creato in modo da prendere in ingresso una classe generica. In questo modo si renderà il codice il più efficiente possibile.

Gestione dei loop

- Problema:

Durante il gioco è possibile finire in dei cicli infiniti per la ricerca della carta corretta: questo avviene per via dei salti, dovuti alle grate, e delle carte mutazione. Bisogna fare in modo di evitare loop infiniti come definito dalle regole del gioco in scatola.

- Soluzione:

Per correggere la possibilità di finire in questo tipo di loop sarà necessario aggiungere un controllo che tenga conto delle carte mutazione per cui si è passati. Tramite questo metodo dotato di memoria sarà possibile rendersi conto dell'imminente loop e si riuscirà a determinare come risultato la carta speciale dove fermare il gioco.

Gestione menu principale:

- Problema:

È presente la necessità di far comparire un menu principale per settare le impostazioni principali del gioco, che deve essere creato prima del *Controller* e fatto in modo da evitare che la *View* contenga parti che non le competono e che creerebbero un accoppiamento tra questa e il resto dell'applicativo, non permettendo un'eventuale modifica futura delle varie parti in modo separato senza intaccare il resto.

- Soluzione:

Si è scelto di creare un popup come menu principale dove far scegliere al giocatore come far eseguire il gioco: il numero di giocatori con i loro nomi e il numero di turni. Questa parte permette di prendere dei dati che poi andranno passati alla creazione del *Controller*. Fornendo all'utente questo popup attraverso il *main* e non tramite la *View* si disaccoppiano le varie parti, infatti potrà partire inizialmente la parte che mostra il menu, a questo punto si avranno le informazioni necessarie alla creazione del *Controller* e una volta fatto ciò si potrà inizializzare la *View* passandole il *Controller* come parametro di input.

Utilizzo di pattern per lo sviluppo del progetto

1. Pattern Static Factory Method (o Simple Factory)

- Problema:

Non essendo banale la creazione del mazzo da 25 carte (che sono composte da 3 caratteristiche prese da degli *Enum* oppure sono carte speciali) ed il suo mescolamento è necessario costruire un metodo per non avere una logica confusionaria all'interno della *Board*. Inoltre per quanto riguarda il lancio dei dadi ci si trova in una situazione simile dovendo estrarre diversi tipi di *Enum*.

- Soluzione:

Per questa complessa istanziazione delle entità di gioco si è ritenuto necessario l'utilizzo del pattern Simple Factory, in particolare implementando metodi statici nelle classi utility *GeneratoreCarte* e *LanciatoreDadi*. Per quanto riguarda la prima, si delega a questa classe la logica complessa di istanziazione, incapsulandola nel metodo *creaMazzoRandom*, invece che lasciare questo compito al *Controller* o alla *Board*. Per la seconda, invece, si garantisce una migliore separazione tra dati pronti all'uso all'interno del *Controller* che li richiede e che potrà ignorare la logica di costruzione interna dei dadi e dati da calcolare tramite logica complessa che sarà svolta nel metodo *lanciaTutti* della classe di utility in questione.

2. Pattern Facade

- Problema:

Bisogna fornire un'interfaccia semplice per un sistema particolarmente complesso, fatto quindi da tante classi.

- Soluzione:

Per avere disaccoppiamento tra l'interfaccia grafica e le singole classi del dominio il *Controller* espone un'interfaccia semplificata ad alto livello, nascondendo la reale complessità del sistema. La UI chiama solo metodi semplici del *Controller* e questo fa da facciata per tutto il *Model*.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Per garantire la robustezza dell'applicativo si è utilizzata la tecnica del testing automatizzato basata sul framework Junit 5. Il testing si è concentrato sulla validazione del *Model* e della logica di dominio, isolandoli dalla *View*. In particolare sono stati svolti test di unità per verificare i comportamenti critici, come il funzionamento della classe *Board* nel calcolo eseguito nel metodo *prossimaPosizione*, controllando la correttezza dell'uso della circolarità del tabellone o per accertare che *GeneratoreCarte* contenesse esattamente 25 carte e avesse la giusta quantità di carte base e carte speciali, testandone così l'integrità.

3.2 Metodologia di lavoro

Il progetto è stato sviluppato interamente in autonomia, quindi il flusso di lavoro è stato abbastanza semplice.

Per prima cosa si è analizzato il problema studiando un UML che potesse comprendere al meglio le classi e le relazioni fra queste da sviluppare. Poi è stato diviso il lavoro in più parti: definizione degli Enum e sviluppo delle classi che costituiscono il *Model*; si è quindi passati allo studio dell'algoritmo per calcolare la soluzione con tutte le accortezze necessarie ed è stata implementata questa parte all'interno del *Controller* insieme a tutte le altre funzionalità di questo componente; ci si è di conseguenza spostati sulla parte più critica, per via delle scarse abilità pregresse, e ci si è cimentati nello studio e nell'implementazione di un'interfaccia grafica semplice ma funzionale; per ultima cosa, dopo uno studio su come gestire separatamente in maniera corretta la *View* rispetto al resto dei componenti, si è aggiunta nel *main* la parte utile per far comparire il menu principale.

3.3 Note di sviluppo

Durante lo sviluppo sono state impiegate diverse funzionalità avanzate del linguaggio Java per rendere il codice più pulito e robusto.

- **Generici Bounded:** all'interno della classe *LanciatoreDadi*, per evitare la duplicazione del codice, per l'estrazione casuale delle facce del dado, è stato implementato il metodo *estraiRandom* che utilizza come parametri in ingresso e in uscita dei tipi generici che però siano limitati al tipo *Enum*.
- **Lambda Expressions:** l'intera interfaccia grafica fa uso delle lambda expressions in modo da mantenere il codice della *View* compatto e leggibile.
- **Algoritmo di rilevamento loop:** è stato sviluppato all'interno del metodo *calcolaSoluzione* un algoritmo per gestire la regola riguardante la “morte” dell'ameba: implementando una logica dotata di memoria si salvano le carte mutazione su cui si è passati in una lista e quindi si è in grado di rilevare cicli infiniti causati dalle combinazioni di grate e mutazioni, interrompendo così volontariamente il loop per selezionare la mutazione che ha “ucciso” l'ameba come soluzione della partita.

4 Capitolo

Commenti finali

4.1 Autovalutazione e lavori futuri

Sono soddisfatta dell'esito del prodotto ottenuto. Ho potuto mettere in pratica le conoscenze e le abilità acquisite durante il corso e anche nel percorso di tirocinio e lavoro. Nonostante non sia la prima volta che mi trovo a sviluppare applicativi di questo genere avevo poca esperienza sulla parte grafica e grazie a questo progetto sono riuscita ad approfondire il lato che è più a contatto con l'esperienza dell'utente, oltre ad aver trovato un nuovo interesse per questa parte che mi era più sconosciuta.

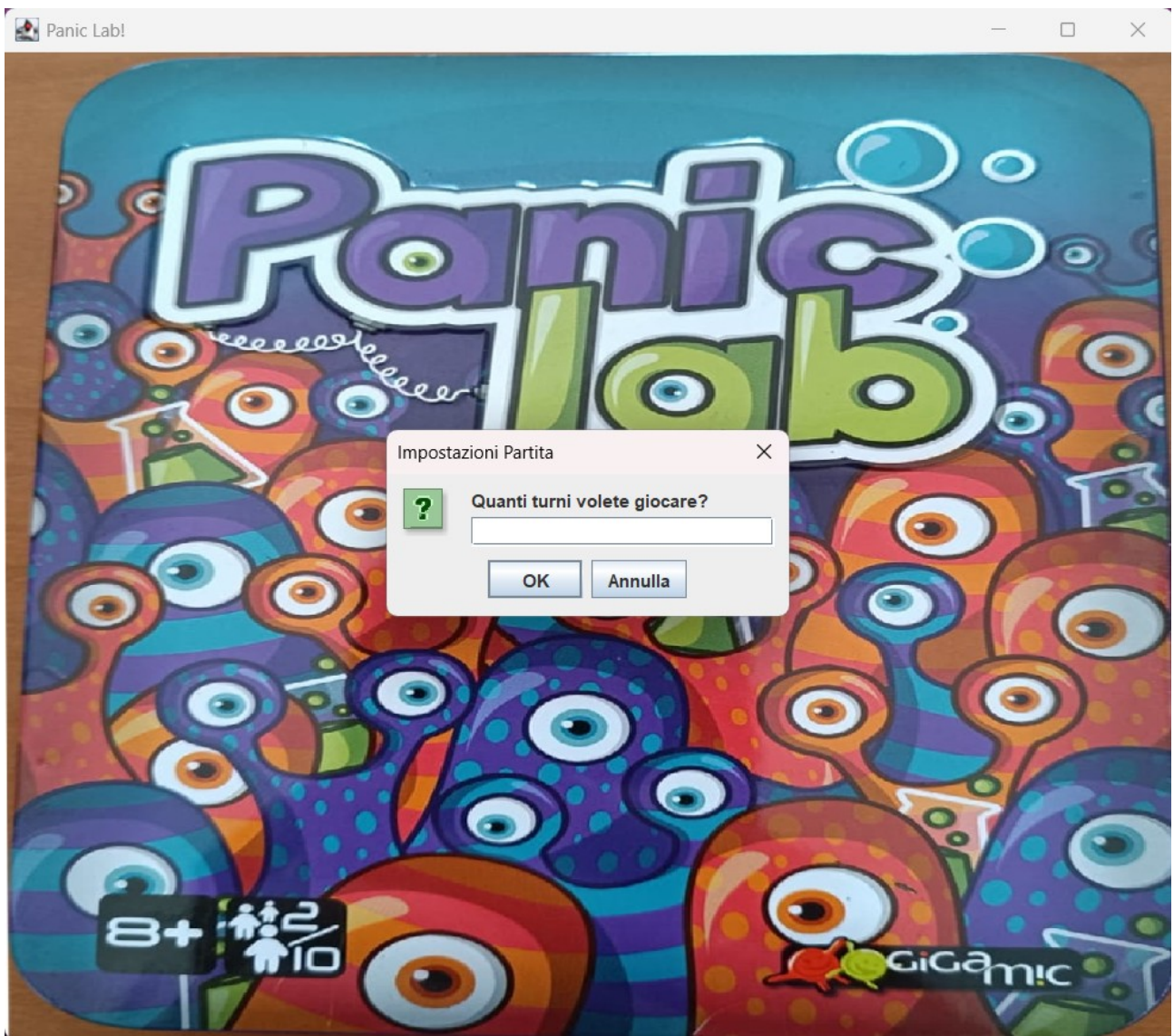
Questo lavoro permette inoltre di avere nuovi sviluppi, perché si potrebbe fare un nuovo studio per aggiungere una parte di storico delle partite, con un eventuale salvataggio del tempo migliore impiegato per trovare la soluzione identificato dal nome del giocatore che l'ha totalizzato. Il progetto è quindi, a mio parere, una risorsa che dopo essere stata creata permette delle espansioni e può quindi essere in continua evoluzione, invece che rimanere un lavoro fermo.

Capitolo 5

Guida utente

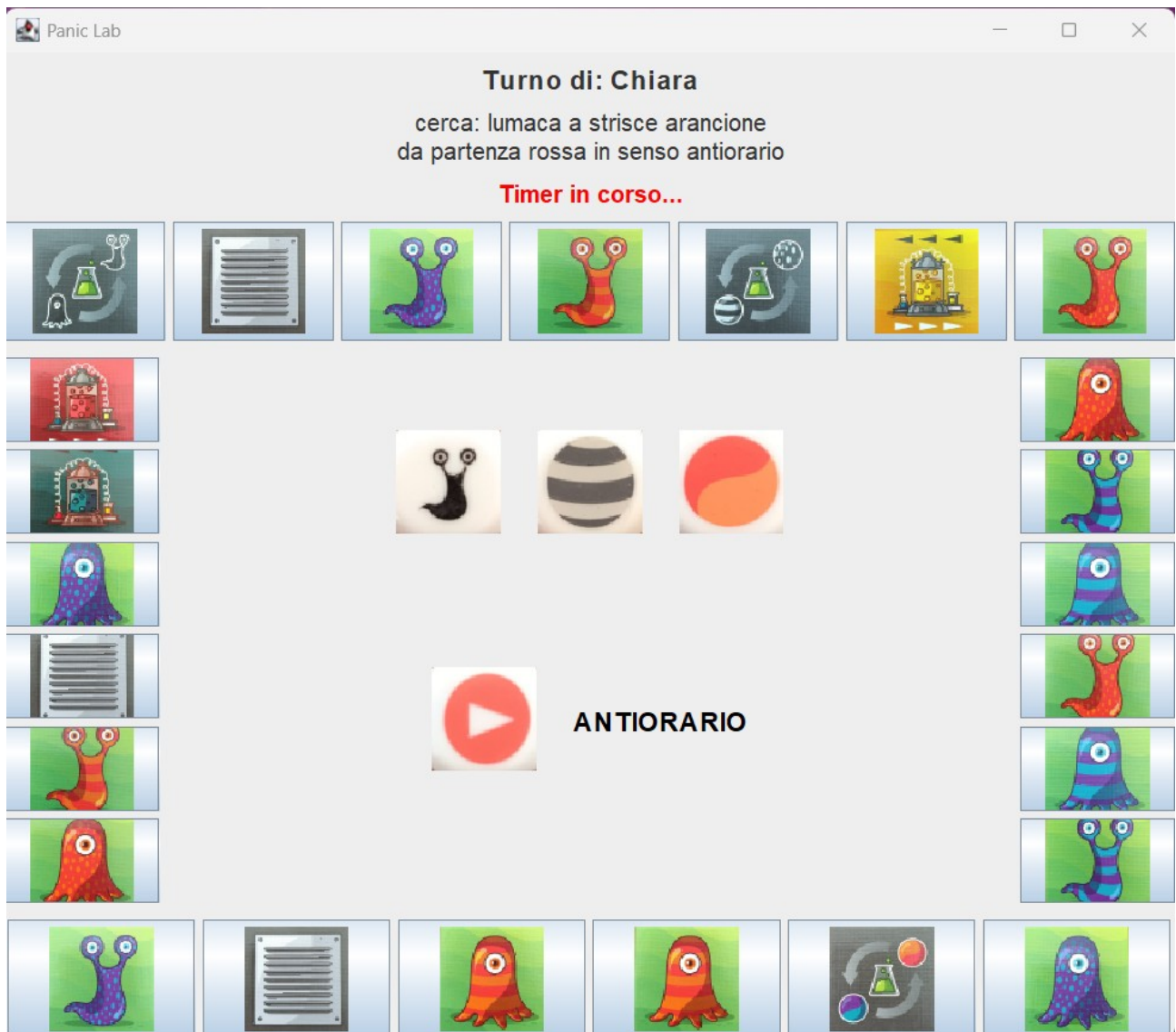
5.1 Menu principale

All'avvio comparirà il menu principale grazie al quale si possono scegliere il numero di turni, di giocatori e i nomi, per poi avviare il gioco.



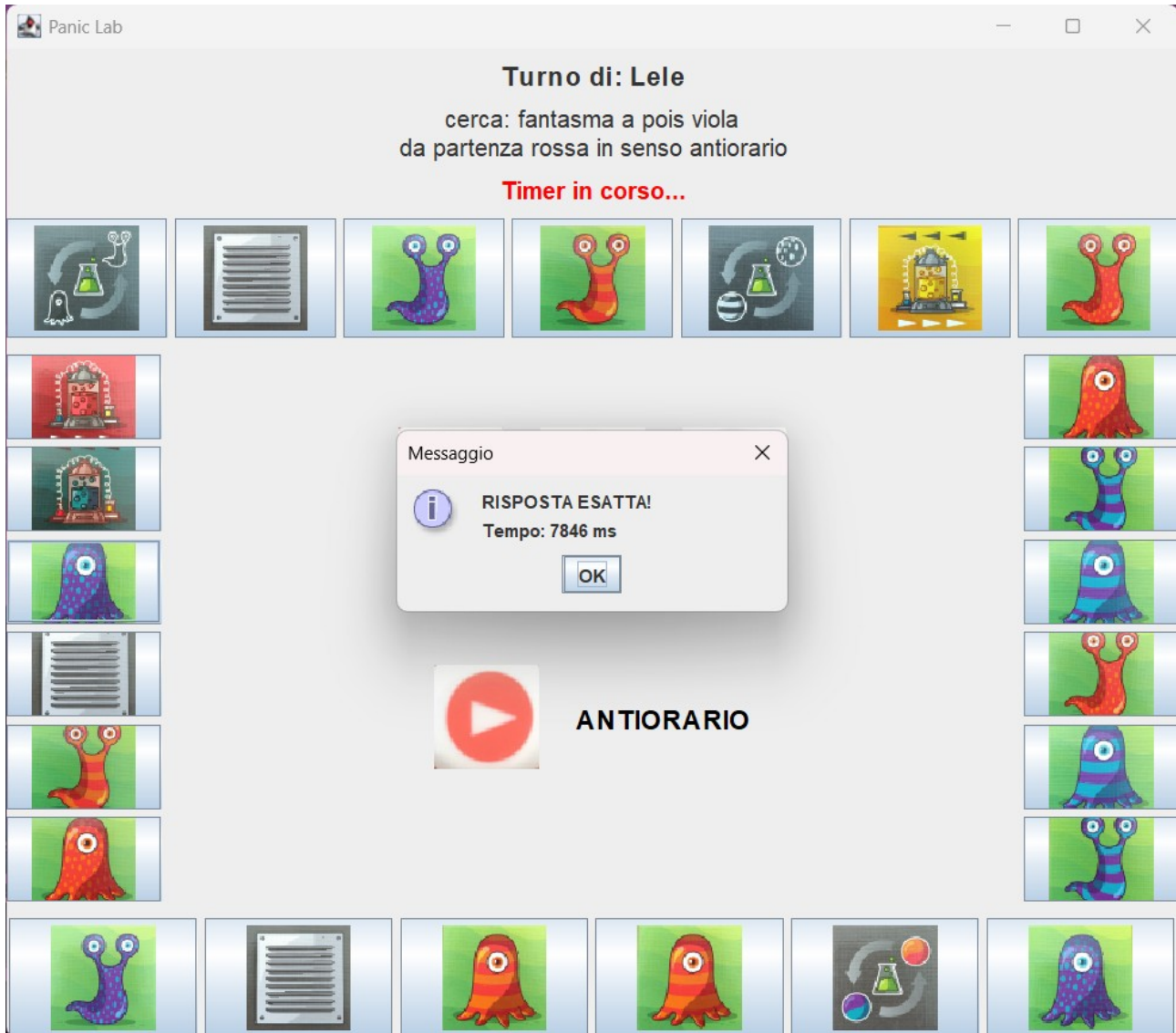
5.2 Inizio partita

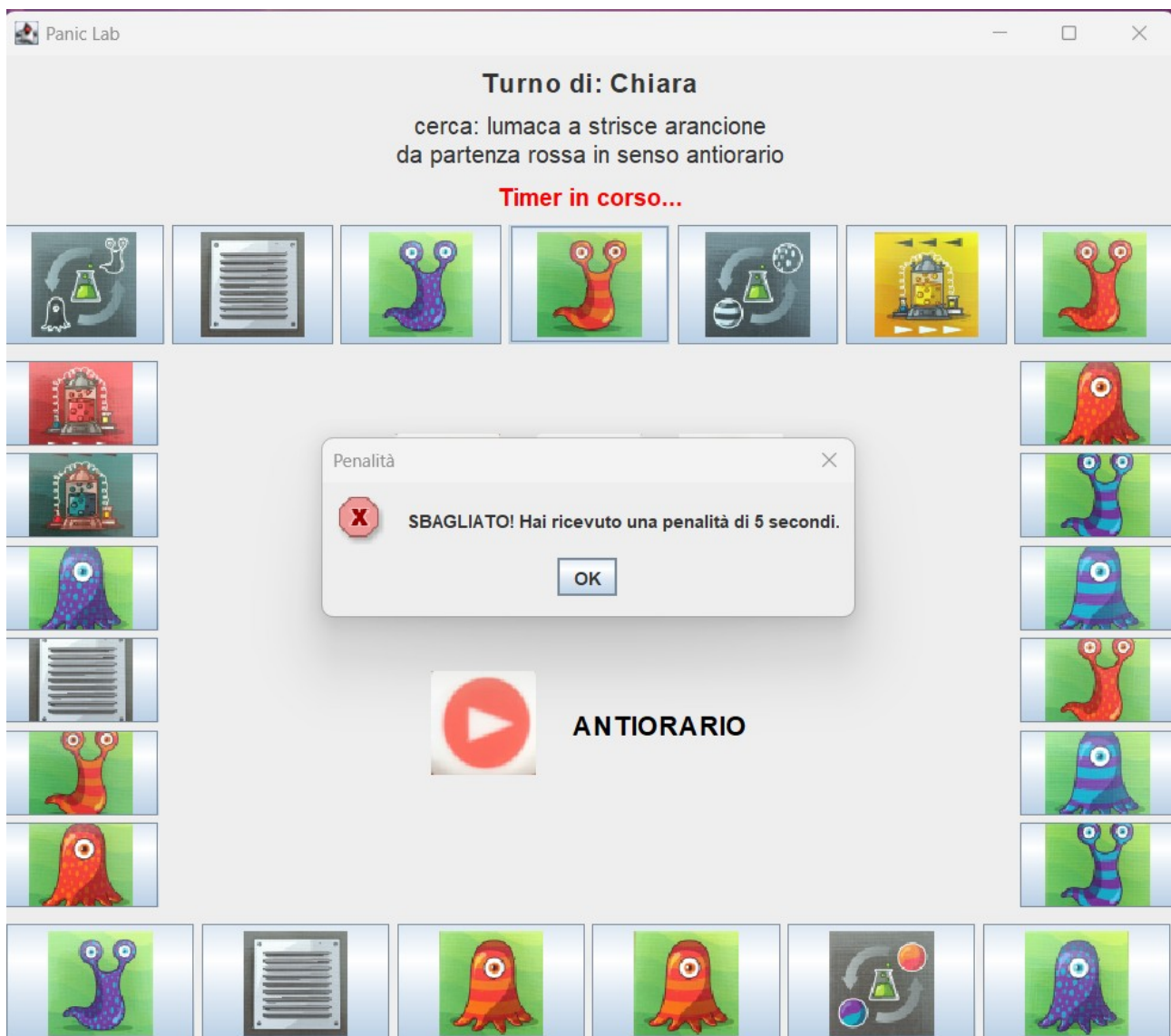
Una volta finite di inserire le impostazioni iniziali compare la schermata di gioco e ha inizio la partita.



5.3 Soluzione

Una volta selezionata una carta comparirà un messaggio per segnalare se la risposta sia giusta o sbagliata.





5.4 Fine partita

Una volta terminati tutti i turni verrà fuori un pannello per mostrare il vincitore con il punteggio totale che gli corrisponde.

